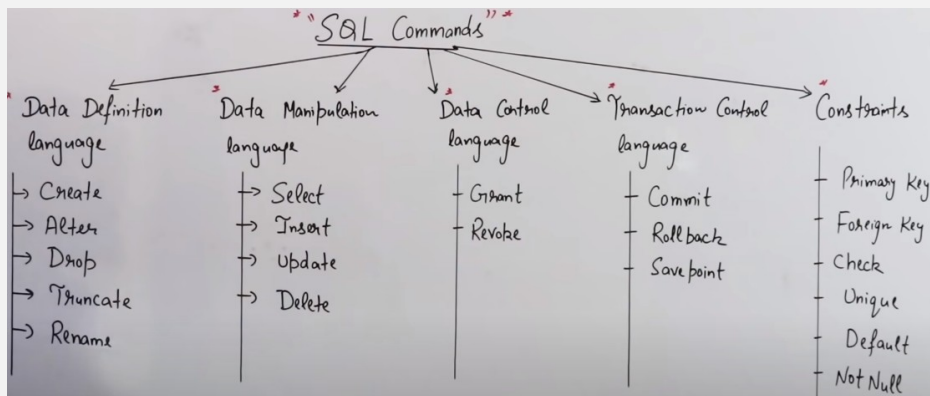


MYSQL COMPLETE PRACTICE



```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql       |
| performance_schema |
| sys        |
+-----+
4 rows in set (0.00 sec)
```

```
mysql> create database nit;
Query OK, 1 row affected (0.01 sec)
```

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql       |
| nit        |
| performance_schema |
| sys        |
+-----+
```

```
mysql> use nit;
Database changed
mysql>
```

create nit db - use nit db -

```
4- create table student(
    name varchar(30),
    id int not null primary key,
    address varchar(50),
    marks int);
```

create student table with column (name, id is not null and also ID will be the primary key, address, marks) - for describe the table use desc

```
mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(30) | YES  |     | NULL    |       |
| id    | int        | NO   | PRI | NULL    |       |
| address | varchar(50) | YES  |     | NULL    |       |
| marks | int        | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Once table created then we need insert the value to the table 2 way - secure way & insecure way

1st query is secure way & 2nd query is insecure way → **(INSERT QUERY)**

```
mysql> insert into student(marks,id,name,address)values(78, 12, 'prakash', 'hyd');
Query OK, 1 row affected (0.01 sec)

mysql> insert into student values('kodi',40,'bng',66);
Query OK, 1 row affected (0.01 sec)
```

User can also insert multiple value in direct way →

```
mysql> insert into student values ('alex',45,'hyd',79), ('cathy',17,'delhi',90),
-> ('dolly',48,'pune',67),('chancy',78,'mumbai',34);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Data stored inside the table & you wanted to see the table then we will use **(SELECT QUERY)**

```
mysql> select * from student;
+-----+-----+-----+-----+
| name   | id   | address | marks |
+-----+-----+-----+-----+
| prakash | 12   | hyd     | 78    |
| cathy   | 17   | delhi   | 90    |
| kodi    | 40   | bng     | 66    |
| alex    | 45   | hyd     | 79    |
| dolly   | 48   | pune    | 67    |
| chancy  | 78   | mumbai  | 34    |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> select name from student;
+-----+
| name |
+-----+
| prakash |
| cathy   |
| kodi    |
| alex    |
| dolly   |
| chancy  |
+-----+
6 rows in set (0.00 sec)

mysql> select name,id from student;
+-----+-----+
| name | id |
+-----+-----+
| prakash | 12 |
| cathy   | 17 |
| kodi    | 40 |
+-----+-----+
```

User need to see the record **WHERE** id =12 & 3 type of select statement

Select the entire table || select the column || select the row by condition or WHERE clause

```
mysql> select * from student where id =12;
+-----+-----+-----+-----+
| name   | id   | address | marks |
+-----+-----+-----+-----+
| prakash | 12   | hyd     | 78    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Once you make ID as primary key you can't insert any value & it will be always unique. ID column never duplicate because we make id as primary key

```
mysql> select * from student;
+-----+-----+-----+-----+
| name   | id   | address | marks |
+-----+-----+-----+-----+
| prakash | 12   | hyd     | 78    |
| cathy   | 17   | delhi   | 90    |
| kodi    | 40   | bng     | 66    |
| alex    | 45   | hyd     | 79    |
| dolly   | 48   | pune    | 67    |
| chancy  | 78   | mumbai  | 34    |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> insert into student values('sam',12,'hyd',56);
ERROR 1062 (23000): Duplicate entry '12' for key 'student.PRIMARY'
mysql>
```

User want to update wherever you find address as Mumbai want to modify Chennai then (**UPDATE**)

```
mysql> update student set address='chennai' where id = 45;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from student;
+-----+-----+-----+-----+
| name   | id   | address | marks |
+-----+-----+-----+-----+
| prakash | 12   | hyd     | 78    |
| cathy   | 17   | delhi   | 90    |
| kodi    | 40   | bng     | 66    |
| alex    | 45   | chennai | 79    |
| dolly   | 48   | pune    | 67    |
| chancy  | 78   | mumbai  | 34    |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

If user want to add any column in existing table → (**ALTER**) but everything is added with null

```
mysql> alter table student add phoneNo int;
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> select * from student;
+-----+-----+-----+-----+-----+
| name | id | address | marks | phoneNo |
+-----+-----+-----+-----+-----+
| prakash | 12 | hyd | 78 | NULL |
| cathy | 17 | delhi | 90 | NULL |
| kodi | 40 | bng | 66 | NULL |
| alex | 45 | chennai | 79 | NULL |
| dolly | 48 | pune | 67 | NULL |
| chancy | 78 | mumbai | 34 | NULL |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> update student set phoneNo=123;
Query OK, 6 rows affected (0.01 sec)
Rows matched: 6 Changed: 6 Warnings: 0
```

```
mysql> select * from student;
+-----+-----+-----+-----+-----+
| name | id | address | marks | phoneNo |
+-----+-----+-----+-----+-----+
| prakash | 12 | hyd | 78 | 123 |
| cathy | 17 | delhi | 90 | 123 |
| kodi | 40 | bng | 66 | 123 |
| alex | 45 | chennai | 79 | 123 |
| dolly | 48 | pune | 67 | 123 |
| chancy | 78 | mumbai | 34 | 123 |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

User want to change specific phoneNo then you need to write the condition

```
mysql> update student set phoneNo=456 where id=12;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> select * from student;
+-----+-----+-----+-----+-----+
| name | id | address | marks | phoneNo |
+-----+-----+-----+-----+-----+
| prakash | 12 | hyd | 78 | 456 |
| cathy | 17 | delhi | 90 | 123 |
+-----+-----+-----+-----+-----+
```

Want to change the column type -

```
mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(30) | YES | | NULL | |
| id | int | NO | PRI | NULL | |
| address | varchar(50) | YES | | NULL | |
| marks | int | YES | | NULL | |
| phoneNo | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> alter table student modify column name varchar(60);
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(60) | YES | | NULL | |
| id | int | NO | PRI | NULL | |
| address | varchar(50) | YES | | NULL | |
| marks | int | YES | | NULL | |
| phoneNo | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

User can drop the column name from the table as well

```
mysql> alter table student drop column phoneNo;
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> select * from student;
+-----+-----+-----+-----+
| name | id | address | marks |
+-----+-----+-----+-----+
| prakash | 12 | hyd | 78 |
| cathy | 17 | delhi | 90 |
+-----+-----+-----+-----+
```

DELATE ANY DATA - DML command & Changes any structure - DDL COMMAND

```
mysql> delete from student where name='kodi';
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select * from student;
+-----+-----+-----+-----+
| name  | id  | address | marks |
+-----+-----+-----+-----+
| prakash | 12 | hyd     | 78    |
| cathy   | 17 | delhi   | 90    |
| alex    | 45 | chennai | 79    |
| dolly   | 48 | pune    | 67    |
| chancy  | 78 | mumbai  | 34    |
+-----+-----+-----+-----+
```

Lets introduce to sql functions →

```
mysql> select sum(marks) from student;
+-----+
| sum(marks) |
+-----+
|          348 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select avg(marks) from student;
+-----+
| avg(marks) |
+-----+
|        69.6000 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select count(name) from student;
+-----+
| count(name) |
+-----+
|           5 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select max(marks) from student;
+-----+
| max(marks) |
+-----+
|          90 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select min(marks) from student;
+-----+
| min(marks) |
+-----+
|          34 |
+-----+
```

```
mysql> select * from student order by marks;
```

```
+-----+-----+-----+-----+
| name  | id  | address | marks |
+-----+-----+-----+-----+
| chancy | 78 | mumbai  | 34    |
| dolly  | 48 | pune    | 67    |
| prakash | 12 | hyd     | 78    |
| alex   | 45 | chennai | 79    |
| cathy  | 17 | delhi   | 90    |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> select * from student order by marks desc;
```

```
+-----+-----+-----+-----+
| name  | id  | address | marks |
+-----+-----+-----+-----+
| cathy  | 17 | delhi   | 90    |
| alex   | 45 | chennai | 79    |
| prakash | 12 | hyd     | 78    |
| dolly  | 48 | pune    | 67    |
| chancy | 78 | mumbai  | 34    |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Wild card character →

Student name which start with a then you need to use wild card character

Pull out the records of student name which start with a ('a%') & names which end with y then'%y')

```
mysql> select * from student where name like 'a%';
+-----+-----+-----+-----+
| name  | id  | address | marks |
+-----+-----+-----+-----+
| alex  | 45 | chennai | 79    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> select * from student where name like '%y';
```

```
+-----+-----+-----+-----+
| name  | id  | address | marks |
+-----+-----+-----+-----+
| cathy  | 17 | delhi   | 90    |
| dolly  | 48 | pune    | 67    |
| chancy | 78 | mumbai  | 34    |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Pull out the records where second alphabet is 'a' then use 2nd wildcard

```
mysql> select * from student where name like '_a%';
```

name	id	address	marks
cathy	17	delhi	90

Pull out the records where second last alphabet is 's' then use below query

```
mysql> select * from student where name like '%s_';
```

name	id	address	marks
prakash	12	hyd	78

Lets create another table to make joins -

```
mysql> create table emp(id int not null primary key,salary int,empcode int,name varchar(30));
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> insert into emp values(12, 20000,102,'aman'),(23,60000,104,'arup'),(78,30000,105,'max'),
-> (80,25000,103,'ram'),(34,90000,106,'sam');
Query OK, 5 rows affected (0.01 sec)
```

```
mysql> select * from emp;
```

id	salary	empcode	name
12	20000	102	aman
23	60000	104	arup
34	90000	106	sam
78	30000	105	max
80	25000	103	ram

Now user has 2 table under nit database || **Innerjoin** - it will display the common records

```
mysql> select * from student;
```

name	id	address	marks
prakash	12	hyd	78
cathy	17	delhi	90
alex	45	chennai	79
dolly	48	pune	67
chancy	78	mumbai	34

5 rows in set (0.00 sec)

```
mysql> select * from emp;
```

id	salary	empcode	name
12	20000	102	aman
23	60000	104	arup
34	90000	106	sam
78	30000	105	max
80	25000	103	ram

5 rows in set (0.00 sec)

```
mysql> select * from student inner join emp on student.id=emp.id;
```

name	id	address	marks	id	salary	empcode	name
prakash	12	hyd	78	12	20000	102	aman
chancy	78	mumbai	34	78	30000	105	max

LEFT JOIN (LEFT OUTER JOIN)→

```
mysql> select * from student;
```

name	id	address	marks
prakash	12	hyd	78
cathy	17	delhi	90
alex	45	chennai	79
dolly	48	pune	67
chancy	78	mumbai	34

5 rows in set (0.00 sec)

```
mysql> select * from emp;
```

id	salary	empcode	name
12	20000	102	aman
23	60000	104	arup
34	90000	106	sam
78	30000	105	max
80	25000	103	ram

5 rows in set (0.00 sec)

```
mysql> select * from student left join emp on student.id=emp.id;
```

name	id	address	marks	id	salary	empcode	name
prakash	12	hyd	78	12	20000	102	aman
cathy	17	delhi	90	NULL	NULL	NULL	NULL
alex	45	chennai	79	NULL	NULL	NULL	NULL
dolly	48	pune	67	NULL	NULL	NULL	NULL
chancy	78	mumbai	34	78	30000	105	max

5 rows in set (0.00 sec)

RIGHT JOIN (RIGHT OUTER JOIN)→

```
mysql> select * from student right join emp on student.id=emp.id;
```

name	id	address	marks	id	salary	empcode	name
prakash	12	hyd	78	12	20000	102	aman
NULL	NULL	NULL	NULL	23	60000	104	arup
NULL	NULL	NULL	NULL	34	90000	106	san
chancy	78	mumbai	34	78	30000	105	max
NULL	NULL	NULL	NULL	80	25000	103	ram

CROSS JOIN →

Meals	MealName	DrinkName
Omlet	Omlet	Orange Juice
Fried Egg	Fried Egg	Orange Juice
Sausage	Sausage	Orange Juice
	Omlet	Tea
	Fried Egg	Tea
	Sausage	Tea
	Omlet	Coffee
	Fried Egg	Coffee
	Sausage	Coffee

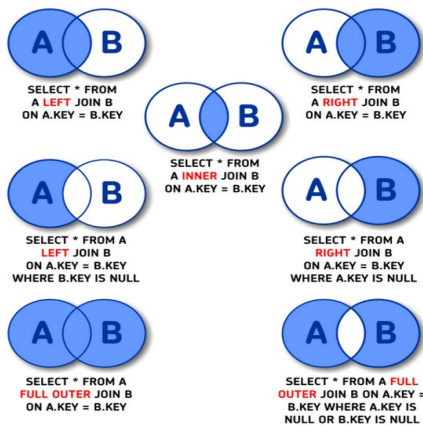
```
mysql> select * from student cross join emp;
```

name	id	address	marks	id	salary	empcode	name
chancy	78	mumbai	34	12	20000	102	aman
dolly	48	pune	67	12	20000	102	aman
alex	45	chennai	79	12	20000	102	aman
cathy	17	delhi	90	12	20000	102	aman
prakash	12	hyd	78	12	20000	102	aman
chancy	78	mumbai	34	23	60000	104	arup
dolly	48	pune	67	23	60000	104	arup
alex	45	chennai	79	23	60000	104	arup
cathy	17	delhi	90	23	60000	104	arup
prakash	12	hyd	78	23	60000	104	arup
chancy	78	mumbai	34	34	90000	106	san
dolly	48	pune	67	34	90000	106	san
alex	45	chennai	79	34	90000	106	san
cathy	17	delhi	90	34	90000	106	san
prakash	12	hyd	78	34	90000	106	san
chancy	78	mumbai	34	78	30000	105	max
dolly	48	pune	67	78	30000	105	max
alex	45	chennai	79	78	30000	105	max
cathy	17	delhi	90	78	30000	105	max
prakash	12	hyd	78	78	30000	105	max
chancy	78	mumbai	34	80	25000	103	ram
dolly	48	pune	67	80	25000	103	ram
alex	45	chennai	79	80	25000	103	ram
cathy	17	delhi	90	80	25000	103	ram
prakash	12	hyd	78	80	25000	103	ram

```
mysql> select * from student inner join emp on student.id = emp.id;
```

name	id	address	marks	id	salary	empcode	name
prakash	12	hyd	78	12	20000	102	aman
chancy	78	mumbai	34	78	30000	105	max

SQL JOINS



```
mysql> show tables;
```

Tables_in_nit
emp
student

SQL TRANSACTION(COMMIT, ROLLBACK, SAVEPOINT) →

Always use START query for sql transaction

Create another student1 table & use below query for this.

```
mysql> insert into student1 values('bin', 12),('bin',12),('bin',12),('bin',12),
('kim',30), ('hari',45), ('rahul',40);
```

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into student1 values('bin1', 78)
-> ;
Query OK, 1 row affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| hari | 45 |
| rahul | 40 |
| bin1 | 78 |
+-----+-----+
8 rows in set (0.00 sec)
```

Below query user delat hari

```
mysql> delete from student1 where name='hari'
Query OK, 1 row affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| rahul | 40 |
| bin1 | 78 |
+-----+-----+
```

Due to some mistake you enter wrong query and delated the records that's why we will do ROLLBACK. If you delated any records from table once use rollback it will give original table. ROLLBACK will act link undo (ctrl+z)

```
mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| rahul | 40 |
| bin1 | 78 |
+-----+-----+
7 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| hari | 45 |
| rahul | 40 |
+-----+-----+
```

```
mysql> insert into student1 values ('welcome', 12);
Query OK, 1 row affected (0.01 sec)

mysql> select * from student1;
+-----+-----+
| name  | id  |
+-----+-----+
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| kim   | 30  |
| hari  | 45  |
| rahul | 40  |
| welcome | 12  |
+-----+-----+
8 rows in set (0.00 sec)
```

Once user define COMMIT & then ROLLBACK will never work

```
mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name  | id  |
+-----+-----+
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| kim   | 30  |
| hari  | 45  |
| rahul | 40  |
| welcome | 12  |
+-----+-----+
8 rows in set (0.00 sec)
```

Lets run below query without commit and records are removed but ROLLBACK is not working

```
mysql> select * from student1;
+-----+-----+
| name  | id  |
+-----+-----+
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| kim   | 30  |
| rahul | 40  |
+-----+-----+
6 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name  | id  |
+-----+-----+
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| bin   | 12  |
| kim   | 30  |
| rahul | 40  |
+-----+-----+
6 rows in set (0.00 sec)
```

ROLLBACK IS NOT WORKING WHY . THAT'S WHY WE NEED TO WRITE 1ST IS start TRANSACTION

Once user mention START TRANSACTION THEN ROLLBACK WILL WORK

When user mention ROLLBACK will work.

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| rahul | 40 |
+-----+-----+
6 rows in set (0.00 sec)

mysql> delete from student1 where name = 'rahul';
Query OK, 1 row affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| rahul | 40 |
+-----+-----+
6 rows in set (0.00 sec)
```

Let's introduce to savepoint → for that again we will start the transaction

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into student1 values('ram',45);
Query OK, 1 row affected (0.01 sec)

mysql> savepoint kamal;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into student1 values('ravi',90);
Query OK, 1 row affected (0.00 sec)

mysql> savepoint prakash;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name | id |
+-----+-----+
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| bin  | 12 |
| kim  | 30 |
| rahul | 40 |
| ram  | 45 |
| ravi | 90 |
+-----+-----+
```

Above query start the transaction and insert value ram and then savepoint to kamal.

After savepoint kamal then add ravi value & save point to prakash. Once you roll back to kamal then you cannot able to see the ravi value

```
+-----+-----+
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| kim   | 30   |
| rahul | 40   |
| ram   | 45   |
| ravi  | 90   |
+-----+-----+
8 rows in set (0.00 sec)

mysql> rollback to kamal;
Query OK, 0 rows affected (0.00 sec)

mysql> select* from student1;
+-----+-----+
| name  | id   |
+-----+-----+
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| kim   | 30   |
| rahul | 40   |
| ram   | 45   |
+-----+-----+
7 rows in set (0.00 sec)
```

Lets take another example of rollback & savepoint

```
mysql> insert into student1 values('suresh',90);
Query OK, 1 row affected (0.00 sec)

mysql> savepoint prakash;
Query OK, 0 rows affected (0.00 sec)

mysql> rollback to prakash;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name  | id   |
+-----+-----+
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| kim   | 30   |
| rahul | 40   |
| ram   | 45   |
| suresh| 90   |
+-----+-----+
8 rows in set (0.00 sec)

mysql> delete from student1 where name = 'suresh';
Query OK, 1 row affected (0.00 sec)

mysql> delete from student1 where name = 'suresh';
Query OK, 1 row affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name  | id   |
+-----+-----+
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| kim   | 30   |
| rahul | 40   |
| ram   | 45   |
+-----+-----+
7 rows in set (0.00 sec)

mysql> rollback to prakash;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from student1;
+-----+-----+
| name  | id   |
+-----+-----+
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| bin   | 12   |
| kim   | 30   |
| rahul | 40   |
| ram   | 45   |
| suresh| 90   |
+-----+-----+
8 rows in set (0.00 sec)
```

LETS UNDERSTAND DDL COMMAND →

```
mysql> create database ddlcommand;
Query OK, 1 row affected (0.02 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| ddlcommand |
| information_schema |
| mysql |
| nit |
| performance_schema |
| sys |
+-----+
6 rows in set (0.00 sec)

mysql> use ddlcommand;
Database changed
mysql>
```

Create command will create the table

```
mysql> create table student(name varchar(30),roll_no int not null primary key, marks int);
Query OK, 0 rows affected (0.04 sec)

mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(30) | YES | | NULL | |
| roll_no | int | NO | PRI | NULL | |
| marks | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> alter table student add addr varchar(50);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(30) | YES | | NULL | |
| roll_no | int | NO | PRI | NULL | |
| marks | int | YES | | NULL | |
| addr | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Alter command will modify the data, delete the data

```
mysql> alter table student drop marks;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(30) | YES | | NULL | |
| roll_no | int | NO | PRI | NULL | |
| addr | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> alter table student modify name varchar(50);
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name | varchar(50) | YES | | NULL | |
| roll_no | int | NO | PRI | NULL | |
| addr | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Using alter command user can change the primary key as well.
Refer to below query.

```
mysql> alter table student drop primary key;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(50)   | YES  |     | NULL    |       |
| roll_no | int          | NO   |     | NULL    |       |
| addr  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> alter table student add primary key(name);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(50)   | NO   | PRI | NULL    |       |
| roll_no | int          | NO   |     | NULL    |       |
| addr  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Alter command also help to rename the table name

```
mysql> rename table student to std;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> desc student;
ERROR 1146 (42S02): Table 'ddlcommand.student' doesn't exist
mysql> desc std;
```

```
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(50)   | NO   | PRI | NULL    |       |
| roll_no | int          | NO   |     | NULL    |       |
| addr  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

TRUNCATE command will act like clear the value from table but table will be display

```
mysql> insert into std values('nit', 6, 'ameerpet');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select * from std;
+-----+-----+-----+
| name | roll_no | addr |
+-----+-----+-----+
| nit  | 6       | ameerpet |
+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> truncate table std;
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> select * from std;
Empty set (0.00 sec)
```

```
mysql> desc std;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(50)   | NO   | PRI | NULL    |       |
| roll_no | int          | NO   |     | NULL    |       |
| addr  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

DROP command will delete the entire table with value

```
mysql> drop table std;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> desc std;
ERROR 1146 (42S02): Table 'ddlcommand.std' doesn't exist
mysql>
```

PRIMARY KEY →

Primary keys serve as unique identifiers for each row in a database table. Foreign keys link data in one table to the data in another table.

This is 1 way to create as primary key

```
mysql> use primarykey;
Database changed
mysql> create table student (id int not null primary key, name varchar(30), age int);
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> desc student;
ERROR 1146 (42S02): Table 'primarykey.student' doesn't exist
mysql> desc student;
```

```
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int          | NO   | PRI | NULL    |       |
| name  | varchar(30)  | YES  |     | NULL    |       |
| age   | int          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2nd way to create primary key as→

```
mysql> create table std(id int not null, name varchar(40), addr varchar(50), primary key(id));
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your
MySQL server version for the right syntax to use near 'std(id int not null, name varchar(40), addr varchar(50), primary key(id))' at
mysql> create table stdent1(id int not null, name varchar(40), addr varchar(50), primary key(id));
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> desc student1;
ERROR 1146 (42S02): Table 'primarykey.student1' doesn't exist
mysql> desc stdent1;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(40)	YES		NULL	
addr	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

How to create primary key to multiple column let see below query

```
mysql> create table student2(id int not null, name varchar(30), age int, constraint pk_student primary key(id, name));
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> desc student2;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(30)	NO	PRI	NULL	
age	int	YES		NULL	

3 rows in set (0.00 sec)

User already created table but forget to assign primary key then use alter command

```
mysql> create table emp(id int, name varchar(30), addr varchar(50));
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> desc emp;
```

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(30)	YES		NULL	
addr	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> alter table emp add primary key (id);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc emp;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(30)	YES		NULL	
addr	varchar(50)	YES		NULL	

```
mysql> create table em(id int, name varchar(20), age int);
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> desc emp;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(30)	YES		NULL	
addr	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> desc em;
```

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(20)	YES		NULL	
age	int	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> alter table em add constraint em primary key(id, age);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc em;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(20)	YES		NULL	
age	int	NO	PRI	NULL	

Remove primary key

```
mysql> alter table em drop primary key;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc em
-> ;
```

Field	Type	Null	Key	Default	Extra
id	int	NO		NULL	
name	varchar(20)	YES		NULL	
age	int	NO		NULL	

3 rows in set (0.00 sec)

FOREIGN KEY

```
mysql> use employee;
Database changed
mysql> create table person(personID int not null primary key,
-> name varchar(30),age int);
Query OK, 0 rows affected (0.03 sec)
```

Let's create another table for reference

```
mysql> desc person;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| personID | int | NO | PRI | NULL | |
| name | varchar(30) | YES | | NULL | |
| age | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> create table personalorder(orderID int not null primary key, ordernumber int, personID int,
-> foreign key(personID) references person(personID));
Query OK, 0 rows affected (0.04 sec)

mysql> desc personalorder;
ERROR 1146 (42S02): Table 'employee.personalorder' doesn't exist
mysql> desc personalorder;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| orderID | int | NO | PRI | NULL | |
| ordernumber | int | YES | | NULL | |
| personID | int | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

LETS CREATE FOREIGN KEY FROM EXISTING TABLE;

```
mysql> create table personorder1(orderID int, ordernumber int, orderperson int);
Query OK, 0 rows affected (0.03 sec)

mysql> show personalorder;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that cor
t syntax to use near 'personalorder' at line 1
mysql> desc student1;
ERROR 1146 (42S02): Table 'employee.student1' doesn't exist
mysql> desc personorder1;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| orderID | int | YES | | NULL | |
| ordernumber | int | YES | | NULL | |
| orderperson | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> alter table personorder1 add foreign key(orderperson)
-> references person(personID);
ERROR 1072 (42000): Key column 'orderperson' doesn't exist in table
mysql> alter table personorder1 add foreign key(orderperson)
-> references person(personID);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc personorder1;
```

Field	Type	Null	Key	Default	Extra
orderID	int	YES		NULL	
ordernumber	int	YES		NULL	
orderpersion	int	YES	MUL	NULL	

```
3 rows in set (0.00 sec)
mysql> desc personorder1;
```

Field	Type	Null	Key	Default	Extra
orderID	int	YES		NULL	
ordernumber	int	YES		NULL	
orderpersion	int	YES	MUL	NULL	

```
3 rows in set (0.00 sec)

mysql> alter table personorder1 add primary key(orderID);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc personorder1;
```

Field	Type	Null	Key	Default	Extra
orderID	int	NO	PRI	NULL	
ordernumber	int	YES		NULL	
orderpersion	int	YES	MUL	NULL	

```
3 rows in set (0.00 sec)
```

SQL JOINS - LEFT JOIN , RIGHT JOIN & CROSS JOIN

SQL join is used to combine rows from two or more table

(INNER) JOIN: Returns records that have matching values in both tables

LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table

RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table

FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table



```
mysql> create database join;
mysql> create table student1(name varchar(30), id int primary key, branch varchar(30));
Query OK, 0 rows affected (0.03 sec)

mysql> create table student2(rollno int primary key, addr varchar(50), phone int);
Query OK, 0 rows affected (0.03 sec)

mysql> show tables;
```

Tables_in_joins
student1
student2

```
mysql> insert into student1 values('john',23,'IT'),('alex',34,'cse'),('ram',13,'cse'),('ravi',14,'me');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> insert into student2 values(23,'delhi',123),(14,'hyd',456),(56,'chen',789),(89,'pune',8976);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> select * from student1;
+-----+-----+-----+
| name | id | branch |
+-----+-----+-----+
| ram  | 13 | cse    |
| ravi | 14 | me     |
| john | 23 | IT     |
| alex | 34 | cse    |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> select * from student2;
+-----+-----+-----+
| rollno | addr | phone |
+-----+-----+-----+
| 14     | hyd  | 456   |
| 23     | delhi | 123   |
| 56     | chen | 789   |
| 89     | pune | 8976  |
+-----+-----+-----+
```

INNER JOIN →

```
mysql> select * from student1 inner join student2 on student1.id = student2.rollno;
+-----+-----+-----+-----+-----+-----+
| name | id | branch | rollno | addr | phone |
+-----+-----+-----+-----+-----+-----+
| ravi | 14 | me     | 14     | hyd  | 456   |
| john | 23 | IT     | 23     | delhi | 123   |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select name, branch, id from student1 inner join student2 on student1.id = student2.rollno;
+-----+-----+-----+
| name | branch | id |
+-----+-----+-----+
| ravi | me     | 14 |
| john | IT     | 23 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select student1.name, student2.rollno from student1 inner join student2 on student1.id = student2.rollno;
+-----+-----+
| name | rollno |
+-----+-----+
| ravi | 14     |
| john | 23     |
+-----+-----+
2 rows in set (0.00 sec)
```

LEFT JOIN →

```
mysql> select * from student1 left join student2 on student1.id=student2.rollno;
+-----+-----+-----+-----+-----+-----+
| name | id | branch | rollno | addr | phone |
+-----+-----+-----+-----+-----+-----+
| ram  | 13 | cse    | NULL   | NULL | NULL   |
| ravi | 14 | me     | 14     | hyd  | 456   |
| john | 23 | IT     | 23     | delhi | 123   |
| alex | 34 | cse    | NULL   | NULL | NULL   |
+-----+-----+-----+-----+-----+-----+
```

RIGHT JOIN →

```
mysql> select * from student1 right join student2 on student1.id=student2.rollno;
+-----+-----+-----+-----+-----+-----+
| name | id | branch | rollno | addr | phone |
+-----+-----+-----+-----+-----+-----+
| ravi | 14 | me     | 14     | hyd  | 456   |
| john | 23 | IT     | 23     | delhi | 123   |
| NULL | NULL | NULL   | 56     | chen | 789   |
| NULL | NULL | NULL   | 89     | pune | 8976  |
+-----+-----+-----+-----+-----+-----+
```

CROSS JOIN →

```
mysql> select * from student1 cross join student2 on student1.id=student2.rollno;
```

name	id	branch	rollno	addr	phone
ravi	14	me	14	hyd	456
john	23	IT	23	delhi	123

```
2 rows in set (0.00 sec)
```

```
mysql> select * from student1 cross join student2;
```

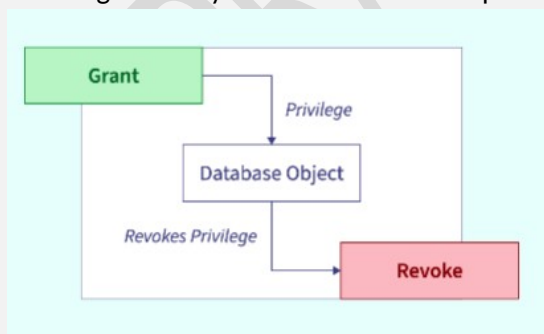
name	id	branch	rollno	addr	phone
alex	34	cse	14	hyd	456
john	23	IT	14	hyd	456
ravi	14	me	14	hyd	456
ram	13	cse	14	hyd	456
alex	34	cse	23	delhi	123
john	23	IT	23	delhi	123
ravi	14	me	23	delhi	123
ram	13	cse	23	delhi	123
alex	34	cse	56	chen	789
john	23	IT	56	chen	789
ravi	14	me	56	chen	789
ram	13	cse	56	chen	789
alex	34	cse	89	pune	8976
john	23	IT	89	pune	8976
ravi	14	me	89	pune	8976
ram	13	cse	89	pune	8976

GRANT & REVOKE → (DO NOT WORK THIS)

In SQL, GRANT and REVOKE are commands used to manage **permissions** (or **privileges**) on database objects like tables, views, procedures, and more. These commands are used to give (grant) or remove (revoke) access to users or roles for performing certain actions (such as SELECT, INSERT, UPDATE, DELETE) on the database.

- **GRANT:** Used to give permissions to users or roles on specific database objects.
- **REVOKE:** Used to remove permissions that were previously granted.

These commands are essential for **managing access control** and **security** in a database system, ensuring that only authorized users can perform certain actions.



```
mysql> select user();
+-----+
| user() |
+-----+
| root@localhost |
+-----+
1 row in set (0.00 sec)

mysql> create user 'ajay'@'localhost' identified by 'ajay';
Query OK, 0 rows affected (0.01 sec)

mysql> show grants for 'ajay'@'localhost';
+-----+
| Grants for ajay@localhost |
+-----+
| GRANT USAGE ON *.* TO `ajay`@`localhost` |
+-----+
1 row in set (0.00 sec)

mysql> grant select on *.* to 'ajay'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> grant insert on employees.student to 'ajay'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```